

Costos del TAD Secuencia – Implementación ListSeq

TP2 – Estructuras de Datos y Algoritmos II

Convenciones

- $n = |s|$: longitud de la secuencia argumento.
- k : índice o cantidad de elementos (para `nthS`, `takeS`, `dropS`).
- $W(f)$, $S(f)$: trabajo y profundidad de la función f pasada como argumento.
- N : suma total de elementos en todas las sub-secuencias (para `joinS`).
- W : costo de **trabajo**.
- S : costo de **profundidad**.

Tabla de costos

Operación	W	S
<code>filterS f s</code>	$O(n \cdot W(f))$	$O(n \cdot S(f))$
<code>reduceS f b s</code>	$O(n \cdot W(f))$	$O(n \cdot S(f))$
<code>scanS f b s</code>	$O(n \cdot W(f))$	$O(n \cdot S(f))$

Implementación

La implementación de `reduceS` en `ArrSeq` utiliza una estrategia de *Divide and Conquer con paralelismo*:

```
reduceS f b (ArrSeq a)
  | A.length a == 0 = b
  | A.length a == 1 = (a A.! 0)
  | otherwise      =
    let (l, r) = (reduceS f b (takeS (ArrSeq a) mid))
              ||| (reduceS f b (dropS (ArrSeq a) mid))
    in f l r
  where
    mid = (A.length a) `div` 2
```

Para un arreglo de n elementos, se divide en dos mitades de tamaño $\lfloor n/2 \rfloor$ y $\lceil n/2 \rceil$, se llama `reduceS en paralelo` sobre ambas (mediante `|||`), y luego se aplica f a los dos resultados. Las operaciones `takeS` y `dropS` sobre `ArrSeq` son $O(1)$ en trabajo y profundidad, ya que utilizan `subArray` (slice del vector subyacente).

Especificación a verificar

La especificación de profundidad (span) para **reduceS** es:

$$S_{\text{reduceS}}(n) = O(S(f) \cdot \log n)$$

Demostración

Recurrencia de profundidad

A partir de los casos base y del caso recursivo se obtiene la siguiente recurrencia:

$$S_{\text{reduceS}}(n) = \begin{cases} O(1) & \text{si } n \leq 1 \\ \max(S_{\text{reduceS}}(\lfloor n/2 \rfloor), S_{\text{reduceS}}(\lceil n/2 \rceil)) + S(f) + O(1) & \text{si } n > 1 \end{cases}$$

El término $S(f)$ corresponde a la aplicación final de f sobre los dos resultados, y $O(1)$ a **takeS/dropS**.

Como ambas mitades tienen tamaño aproximadamente $n/2$, la recurrencia se simplifica a:

$$S_{\text{reduceS}}(n) = S_{\text{reduceS}}(\lfloor n/2 \rfloor) + S(f) + O(1)$$

Resolución de la recurrencia

Suponiendo $n = 2^k$ y desenrollando la recurrencia:

$$\begin{aligned} S_{\text{reduceS}}(n) &= S_{\text{reduceS}}\left(\frac{n}{2}\right) + S(f) + O(1) \\ &= S_{\text{reduceS}}\left(\frac{n}{4}\right) + 2 \cdot S(f) + O(1) \\ &\vdots \\ &= S_{\text{reduceS}}(1) + k \cdot S(f) + O(1) \\ &= O(1) + S(f) \cdot \log_2 n + O(1) \end{aligned}$$

Por lo tanto:

$$\boxed{S_{\text{reduceS}}(n) = O(S(f) \cdot \log n)}$$

Interpretación

El árbol de llamadas tiene altura $\log_2 n$. En cada nivel, ambas ramas se ejecutan en paralelo, por lo que la profundidad de cada nivel es $S(f) + O(1)$. Al acumular los $\log_2 n$ niveles se obtiene la profundidad total $O(S(f) \cdot \log n)$, verificando la especificación.